01
Software Quality: overview and techniques (principles, rules, agility, patterns)

Mirko Viroli
mirko.viroli@unibo.it

C.D.L. Magistrale in Ingegneria e Scienze Informatiche
Alma Mater Studiorum—Università di Bologna, Cesena

a.a. 2025/2026

Outline

1. Good design, abstraction, and process
2. Software quality criteria
3. Quality framework: problems and SE principles
4. Quality framework: OO paradigm
5. Software methodologies and the agile manifesto
6. Agile unit testing
7. A simple TDD-based example
8. Quality framework: (OO) design principles
9. Refactoring techniques and design patterns
10. Quality and the language paradigm

Software is key

Software accounts for a big share of world economy

- estimated $\sim$ 15 percent of global GDP (software industry's indirect impact)
- estimated $\sim$ 30 percent of global GDP in 2030 for digital economy

Producing good software is costly

- productivity is $\ll$ 100 "lines of code" per day per person (whole life-cycle)

Generative AI is speeding up software development and overall impact of software

The need of a software process

Problems with developing software

- software tends to be very complex
- pieces of software are often easy to "somewhat prototype"
- software is error prone, and often very difficult to debug
- the time from prototype to correct deployed software can be huge

$\Rightarrow$ software projects can and will simply fail

Learning from experience

- what is the software "quality" that leads to successful projects?
- what are good development process and programming techniques to promote quality?

Steps of software development (not strictly in waterfall)

Requirement engineering (elicitation, analysis, specification)

- understanding the problem, by gathering requirements and understanding/analysing/organising/formalising them

Design

- defining the key elements of the solution: a software system
- by incremental refinement: from architecture to details

Implementation

- actual development: choosing technologies and coding

Post-implementation

- testing, deployment, maintenance: the resource-demanding phase!

“Frequently forgotten facts about SoftEng” R.Glass

Complexity and people

- 10% increase in problem complexity leads to 100% in solution
- good programmers are up to 30 times better than mediocre ones

Requirements and design

- two causes of project runaway:
  instability of requirements + optimistic estimation

Costs

- error detection and removal is 40% of development
- rigorous review removes 90% of errors
- maintenance consumes from 40% to 80% of overall cost

The importance of quality software

What problems can quality software address?

1. better adapting a software to instability of requirements
2. reduce maintenance costs
3. support detection and removal of errors

Achieving quality software

- understanding what quality software is (criteria and metrics)
- using programming practice known to lead to quality software
- well organising the whole development process

⇒ namely, the goals of this course

Eventually understanding that...

...being able to write very high quality software, means to have a wide set of key skills (technical, organizational) the most important of which is:

⇒ high ability in capturing and managing "abstraction"

On the abstraction principle

Abstraction

the single concept behind a variety of pragmatic concretions/individuals

As a philosophical concept

The (alleged) process in concept formation of recognizing some set of common features in individuals, and on that basis forming a concept of those features.

Most specifically in Software Engineering

- Mitigate opacity: code directly captures the idea, it is not bloated with unnecessary details
- DRY principle: Don't Repeat Yourself – now or in the future
- DIP principle (Dependency inversion): depend on abstraction (interfaces), not on implementation details (classes)
- ...

Outline

1. Good design, abstraction, and process
2. Software quality criteria
3. Quality framework: problems and SE principles
4. Quality framework: OO paradigm
5. Software methodologies and the agile manifesto
6. Agile unit testing
7. A simple TDD-based example
8. Quality framework: (OO) design principles
9. Refactoring techniques and design patterns
10. Quality and the language paradigm

SQ definitions and measurements

Definitions of SQ

- well translating future needs into measurable characteristics (Deming)
- meeting the customer needs + with freedom from deficiencies (Juran)
- perspectives:
  - transcendental (philosophical, promoting elegance and coherence)
  - product (quantifiable and measurable characteristics)
  - manufacturing (conforming to agreed requirements)
  - user (satisfying the users of the produced software)

Measurements

- reliability: resiliency to changes and structural solidity
- efficiency: good performance and scalability
- security: no security breaches
- maintainability: adaptability, evolvability, control
- size

Internal vs external quality

External Quality

- determines the fulfillment of stakeholder requirements
- Correctness, Usability, Efficiency, Reliability, Integrity, Adaptability, Accuracy, and Robustness.

Internal Quality

- determines your ability to move forward on a project
- Maintainability, Flexibility, Portability, Re-usability, Readability, Testability, and Understandability.

Some fact

- people typically perceive "external quality"
- "internal quality" is the key means to "external quality"
- modern approaches require internal features to be readily observable
- aiming at external quality bypassing internal one is often the problem

Code quality

Software quality vs code quality

- software quality is a combination of:
  - code/data/process/tools/platforms/library quality
- code quality is a necessary precondition to software quality
- code quality focusses on the shape of code and how code can be evolved so as to achieve overall internal quality, then impacting external quality

Technical debt

Definition

the extra development work that arises when code that is easy to implement in the short run is used instead of applying the best overall solution

Basically the "patch" problem

- one quickly and directly solves the problem, knowing it is sub-optimal
- because of pressure (or other reasons), it delays improvement of the solution
- soon, this is going to cost (typically, more than fixing immediately)
- considered "ultimate evil" in other areas of engineering

⇒ sometimes, technical debt is necessary or unavoidable, though

Overall

- it is a measure of distance to "complete quality"
- almost impossible to measure, should mostly proceed empirically

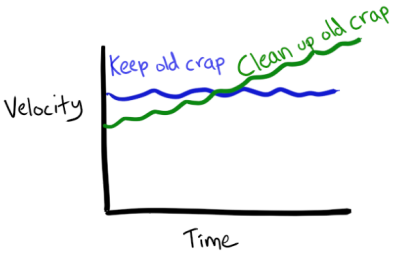
A simple example of technical debt situation

```java
import java.nio.file.Files;
import java.nio.file.Paths;
import java.util.Collections;
import java.util.List;

public class Configuration {

    private List<String> data;

    public void loadFromFile(final String fileName){
        try{
            this.data = Files.readAllLines(Paths.get(fileName));
        } catch (IOException e){
            System.out.println(e); // Technical debt here!
        }
    }

    public List<String> getData(){
        return Collections.unmodifiableList(this.data);
    }

}
```

Notes on technical debt

Walter Wise – www.business.com

Good quality software and code does not happen by accident. It happens as a result of consistent and passionate focus by the software implementation team to meet quality goals. This is something that is very important for your software project. Some developers have a habit of side-stepping quality concerns; especially when they are being pushed for short-term results.



https://www.exact.com, Eric Ammerlaan

Code smell

A definition

any symptom, surface indication in the source code of a program that possibly indicates a deeper problem, violating fundamental design principles and negatively impacting the quality of implementation

Facts

- quintessential examples (to be detailed):
  code duplication, bad names, abstraction "jumps", partial implementations, simple OO principles violation
- they are not necessarily bugs
- they typically create technical debt
- often they are present since system creation
- they typically badly impact quality and hence costs

Surface vs deep problems

- note that many problems in coding are surface ones, other are however more profound and difficult to address
- they typically amount to general software engineering problems

Code smells. Example 1: neglecting code conventions

Code conventions

- a language provides a syntax, but on top of it there are typically additional conventions
  - style of names, spacing/formatting, obsolete constructs not to use, structure of programs
- some language comes with standard conventions
- they change across language (see C# vs Java)
- subgroups tend to add further constraints (not remove them!)
- it is key to follow them all, as they simplify the life of colleagues
- simple examples in Java
  - never prefix interfaces (`I`) variables, or anything else
  - camel case for all identifiers
  - never declare two variables in the same line
  - do not abuse obscure operators (`+=`, `--`)

Code smells. Example 2: not adhering to modern rules of effectiveness

Inspired by the collection in "Effective Java", by J.Bloch

- know and use your libraries
- minimize accessibility of class members
- minimize mutability
- favour composition over inheritance
- design and document for inheritance, or prohibit it
- prefer interfaces over abstract classes
- check parameters for their validity in public APIs
- document all public APIs
- make defensive copies when needed
- minimise the scope of local variables
- prefer for-each loops
- avoid floating point when exact values are needed
- avoid strings where other types are more appropriate
- program against interfaces, not classes
- ...

Code smells. Example 3: not using clean coding techniques

General rules of clean coding

- code should be as readable as good prose
- code should always reveal the intent, never obfuscate it
- code should be clear to all programmers, skilled or not

Good code: meaningful names

Recipes

- use intention-revealing names:
  ⇒ `elapsedTimeInDays` is better than `d`, even for a variable
- use pronounceable names, not acronyms or pseudo-acronyms
  ⇒ "`class DtaRcrd102 {..}`"? no thanks
- use searchable names (see also magic numbers/constants)
  ⇒ `MAX_CLASSES_PER_STUDENT` equals to 7 is a wonderful thing
- no prefixes, in vars or interfaces (no `I` in front) !!
  ⇒ `interface IScheduler {..}` is ugly... recall interfaces are key
- class names are names, or phrase names
  ⇒ not verbs like "`Processing`", but rather "`TransactionProcessor`"
  ⇒ plurals are sometimes used for helper classes, like "`Objects`" or "Collections"
- function names are typically phrase verbs
  ⇒ better use factories not constructors
  ⇒ functions that just read elements are sometimes names
  ⇒ mathematical functions are one-character following the tradition

An example: spot the inadequacies!

```java
// A modelling abstraction for a DVCS
public interface Model {

    void setCmd();

    void doCopy(String path2);

    void setRepository(String repo);

    boolean check_Existence();

    void doCommit(String commento);

    String[] Log();
}
```

Good code: functions and methods

Recipes

- should be small (10 lines), simple, and quickly understandable
  $\Rightarrow$ factor out what needed in private methods
- do one thing and only there (will see SRP and DRY principles)
- work at one level of abstraction (will see DIP and ISP principles)
  $\Rightarrow$ e.g., don't mix string manipulation with other stuff
- organise them top to down, like they were a story
- it is ok to have a long name wrt a short obscure one
- 0,1 or 2 arguments, avoid unordering
  $\Rightarrow$ consider wrapping more arguments in classes/builders
- have no side-effects unless is the only thing. . . as in setters
- avoiding eccessive proliferation is always a secondary issue

A (non-)clean code example...

```java
    public static String testableHtml(PageData pageData, boolean includeSuiteSetup) throws Exception {
        WikiPage wikiPage = pageData.getWikiPage();
        StringBuffer buffer = new StringBuffer();
        if (pageData.hasAttribute("Test")) {
            if (includeSuiteSetup) {
                WikiPage setup = PageCrawlerImpl.getInheritedPage(Responder.SETUP_NAME, wikiPage);
                if (setup != null) {
                    WikiPagePath pagePath = setup.getPageCrawler().getFullPath(setup);
                    String pagePathName = PathParser.render(pagePath);
                    buffer.append("!include -setup .").append(pagePathName).append("\n");
                }
            }
            WikiPage setup = PageCrawlerImpl.getInheritedPage("SetUp", wikiPage);
            if (setup != null) {
                WikiPagePath setupPath = wikiPage.getPageCrawler().getFullPath(setup);
                String setupPathName = PathParser.render(setupPath);
                buffer.append("!include -setup .").append(setupPathName).append("\n");
            }
        }
        buffer.append(pageData.getContent());
        if (pageData.hasAttribute("Test")) {
            WikiPage teardown = PageCrawlerImpl.getInheritedPage("TearDown", wikiPage);
            if (teardown != null) {
                WikiPagePath tearDownPath = wikiPage.getPageCrawler().getFullPath(teardown);
                String tearDownPathName = PathParser.render(tearDownPath);
                buffer.append("\n").append("!include -teardown .").append(tearDownPathName).append("\n");
            }
            if (includeSuiteSetup) {
                WikiPage tearDown = PageCrawlerImpl.getInheritedPage(Responder.TEARDOWN_NAME, wikiPage);
                if (tearDown != null) {
                    WikiPagePath pagePath = tearDown.getPageCrawler().getFullPath(tearDown);
                    String pagePathName = PathParser.render(pagePath);
                    buffer.append("!include -teardown .").append(pagePathName).append("\n");
                }
            }
        }
        return buffer.toString();
```

An improved version (1/3)

```java
public class SetupTeardownIncluder {
    private PageData pageData;
    private boolean isSuite;
    private WikiPage testPage;
    private StringBuffer newPageContent;
    private PageCrawler pageCrawler;

    public static String render(PageData pageData) throws Exception {
        return render(pageData, false);
    }

    public static String render(PageData pageData, boolean isSuite) throws Exception {
        return new SetupTeardownIncluder(pageData).render(isSuite);
    }

    private SetupTeardownIncluder(PageData pageData) {
        this.pageData = pageData;
        testPage = pageData.getWikiPage();
        pageCrawler = testPage.getPageCrawler();
        newPageContent = new StringBuffer();
    }

    private String render(boolean isSuite) throws Exception {
        this.isSuite = isSuite;
        if (isTestPage()) {
            includeSetupAndTeardownPages();
        }
        return pageData.getHtml();
    }

    private boolean isTestPage() throws Exception {
        return pageData.hasAttribute("Test");
    }
```

An improved version (2/3)

```java
    private void includeSetupAndTeardownPages() throws Exception {
        includeSetupPages();
        includePageContent();
        includeTeardownPages();
        updatePageContent();
    }

    private void includeSetupPages() throws Exception {
        if (isSuite) {
            includeSuiteSetupPage();
        }
        includeSetupPage();
    }

    private void includeSuiteSetupPage() throws Exception {
        include(SuiteResponder.SUITE_SETUP_NAME, "-setup");
    }

    private void includeSetupPage() throws Exception {
        include("SetUp", "-setup");
    }

    private void includePageContent() throws Exception {
        newPageContent.append(pageData.getContent());
    }

    private void includeTeardownPages() throws Exception {
        includeTeardownPage();
        if (isSuite) {
            includeSuiteTeardownPage();
        }
    }
```

An improved version (3/3)

```java
    private void includeTeardownPage() throws Exception {
        include("TearDown", "-teardown");
    }

    private void includeSuiteTeardownPage() throws Exception {
        include(SuiteResponder.SUITE_TEARDOWN_NAME, "-teardown");
    }

    private void updatePageContent() throws Exception {
        pageData.setContent(newPageContent.toString());
    }

    private void include(String pageName, String arg) throws Exception {
        WikiPage inheritedPage = findInheritedPage(pageName);
        if (inheritedPage != null) {
            String pagePathName = getPathNameForPage(inheritedPage);
            buildIncludeDirective(pagePathName, arg);
        }
    }

    private WikiPage findInheritedPage(String pageName) throws Exception {
        return PageCrawlerImpl.getInheritedPage(pageName, testPage);
    }

    private String getPathNameForPage(WikiPage page) throws Exception {
        WikiPagePath pagePath = pageCrawler.getFullPath(page);
        return PathParser.render(pagePath);
    }

    private void buildIncludeDirective(String pagePathName, String arg) {
        newPageContent.append("\n!include ")
                      .append(arg)
                      .append(" .")
                      .append(pagePathName)
                      .append("\n");
    }
}
```

Good code: comments and formatting

Recipes for comments

- don't use them! they tend to integrate bad code, better rewrite code
- do not repeat what the code could already say if well structured
- javadoc public APIs, not private things. . . do it for interfaces
- never comment code out, never section methods, don't use as noise

Recipes for formatting

- adhere to your group's practice, which should constrain the community standard, not open it. . .
- blank lines to separate concepts, keep close concepts close
- dependent functions below
- 100-120 chars per line, be careful on splitting

Again on "Why good code matters"...

Bad code and consequences

- bad code leads to more time to maintenance (fixing bugs/extend/change)
- Leblanc's law "Later equals never"
  `http://on-agile.blogspot.it/2007/04/why-you-wont-fix-it-later.html`
- why people write bad code (and hence accumulate technical debt):
  - no competence
  - want to show they are quicker than actually should be
  - skilled, but with wrong understanding of what is good code or why it matters

Good code

- fully tested, integrated, reviewed, refactored and readable
- elegant, simple, few dependencies, written with care, reads like prose
- boy scout attitude: "leave the campground cleaner than you found it"

Philosophic view

- fight to promote the idea that coding is "creating beautiful abstractions"
- your life at work will just be better...

Outline

1. Good design, abstraction, and process
2. Software quality criteria
3. Quality framework: problems and SE principles
4. Quality framework: OO paradigm
5. Software methodologies and the agile manifesto
6. Agile unit testing
7. A simple TDD-based example
8. Quality framework: (OO) design principles
9. Refactoring techniques and design patterns
10. Quality and the language paradigm

A code quality framework

Code defects

Needless repetitions and complexity, rigidity, fragility, immobility, viscosity, opacity

Software engineering principles

Rigour/formality, separation of concerns, modularity, abstraction, anticipation of change, generality, incrementality

Principles of good (OOP) programming

(KISS, DRY) + (SRP, OCP, LSP, ISP, DIP) + ...

Design patterns

Template recipes to inject good principles in design (GoF patterns, but also others)

Methology

- Waterfall: rigorous management of requirements-to-implementation transition
- Agility: test-oriented development, continuous integration, advanced tools, agile development processes
- Test-driven development (TDD)

Problems in software

Defects

- needless repetition (DRY)
  - bugs tend to multiplicate due to a single cause
- needless complexity (KISS)
  - code is more difficult to understand, modify, and test
- rigidity — the design is hard to change
  - small changes cause many cascade modifications
  - ⇒ reluctance to change and underestimation of costs
- fragility — the design is easy to break
  - changes in one module introduce unexpected errors or problems elsewhere
  - ⇒ times is spent in finding bugs, fragile modules are often in the buglist
- immobility — the design is hard to reuse
  - code seems general but cannot be reused without high cost and risks
- viscosity — proper changes tend to be costly
  - slow or unefficient development/testing (tools are key!)
  - changes were often performed by hacks (e.g., pervasive technical debts)
- opacity — modules are difficult to understand
  - high effort to keep the code and its intent understandable

Defects of software I

Needless repetition

- a piece of code is one or multiple times cut & paste & modified as needed

⇒ bugs and modifications tend to multiplicate due to a single cause, until fixing problems becomes simply impractical

- large-scale examples include
  - multiple forms in GUIs
  - DB access logics (queries, checks)
  - large and variable models (many entities with small differences)
  - highly configurable pieces of software (software product lines)
- small-scale examples include
  - actually any resemblance in any two parts of the code

Defects of software II

Needless complexity

- solutions that are more complex / involved / sophisticated than needed are used

⇒ code become more difficult to understand, modify, and test than expected

- examples include
  - reinventing the wheel when a library exists that do the same
  - of more solutions, adopt the trickiest one
  - additional cyclomatic complexity due to small competence
  - using optimisation techniques where not needed
  - supporting requirements no one asked
- the good approach
  - be unsatisfied of any piece of code more complex than your experience tells it should be

Defects of software III

Rigidity

- the design is hard to change
- small changes cause many cascade modifications
- performing changes tend to cost more than expected

⇒ generally induces reluctance to change and underestimation of modification costs

- examples include
  - repetition without common factorisation
  - poor use of information hiding and encapsulation
  - too tight dependency from lower-level implementations
- typical reason
  - too much unneeded "dependency" between modules

Defects of software IV

Fragility

- the design is easy to break
- changes in one module introduce errors or problems elsewhere

⇒ time is spent in finding bugs, fragile modules are often in the buglist

- examples include
  - modules that make too much (undocumented) assumptions on another
  - modules that provide partial implementations, that do not cover all cases well
  - modules are highly coupled, but in an indirect way
  - solutions are so complicated that are not really understood, and turn out to fail

Defects of software V

Immobility

- the design is hard to reuse

⇒ code seems general but cannot be reused without high cost and risks

- examples include
  - configurable details hidden in implementation
  - poorly structured interfaces of complex components
  - software do not reveal generality and commonalities
  - direct use of implementation details

Defects of software VI

Viscosity (of design and environment)

- changes tend to be implemented by hacks
- slow or inefficient development environments

⇒ psychological acceptance of low quality (e.g., pervasive technical debts)

- examples include
  - the IDE makes compilation/execution slow/uneffective
  - automated tests are prevented or uneffective
  - deployment is very complex, hence acceptance test is uneffective
  - design does not allow easy extension of software
  - existing hacks and patch keep promoting new ones

Defects of software VII

Opacity

- code is difficult to understand

⇒ high effort to keep the code and its intent understandable, abuse of inline comments

- examples include
  - bad organisation of modules
  - bad names to modules/functions/variables
  - types and structure that does not reveal intent
  - obscure formatting of flow-based code

Repetitive code

```java
private Enemy createEnemy() {
  int type = rand.nextInt(2);
  if (type == 0) {
    return new Cactus(800, cactus1.getWidth() - 10,
                          cactus1.getHeight() - 10, cactus1);
  } else {
    return new Cactus(800, cactus2.getWidth() - 10,
                          cactus2.getHeight() - 10, cactus2);
  }
}
```

Complex code

```
public class StatValuesImpl implements StatValues {
    private static final int NO_MAX = -1;
    private int actual;
    private in max;

    public int getActual {
        return actual;
    }

    public int getMax {
        return max;
    }

    public void updateActual(int value) {
        if (actual + value <= 0){
            actual = 0;
        } else {
            if (max != NO_MAX && actual + value > max){
                actual = max;
            } else {
                actual = actual + value;
            }
        }
    }
}
```

Software engineering: principles

Principles

- rigour and formality (RF)
- separation of concerns (SOC)
- modularity (MOD)
- abstraction (ABS)
- anticipation of change (AOC)
- generality (GEN)
- incrementality (INC)

General facts

- we all know them, we all – at least in principle – concur they should be used
- sometimes we simply believe we have no time to follow them
- in reality, one should judiciously know when to apply each single of them

Software engineering: principles I

RF – rigour and formality

- it means to describe/enact some behaviour as precisely as mathematics would do
- it is key where errors would incur in high technical debt afterwards
- this is more likely applied by developers with scientific background
- this is reasonably done only on critical/small subparts of code
- rigorous descriptions can be provided in tests
- rigorous descriptions can be provided as code documentation
- examples include
  - use of pseudo-mathematical structures: finite state-machines, petri-nets, other automata
  - use of advanced/new algorithms where needed
  - use of DSLs or DSL-like APIs to configure complex aspects
  - use of rule-like specification in tests

Software engineering: principles II

SOC – separation of concerns

- master complexity by grouping issues, and consider each in isolation
- recognises the need for human beings to work within a limited context
- this is a very general principle, some others are sort of instances of it
- examples include
  - divide the projects in increments, to be developed one after the other
  - split developers in teams, split work within a team
  - divide tests into unit / integration / acceptance / stress
  - divide requirements into functional / non-functional
  - consider correctness of a program independently of performance

Software engineering: principles III

MOD – modularity

- divide software in modules, intended as groups of basic abstractions (classes, sources, other modules)
- seek for high cohesion within a module, low coupling across modules
- allocate tasks/aspects to modules (basically adhering to SOC)
- modularise at multiple levels, proceeding top-down or bottom-up
- define modules so as to support reusability, composability
- examples include
  - structure of packages/namespaces
  - nesting of classes (or other constructs in other languages)
  - structure of deployment packages (object files, DLLs, JARs, ZIPs)

Software engineering: principles IV

ABS – abstraction

- in design, identify the key aspects to be considered and ignore others, treated as unnecessary details
- applicable to programming constructs at various levels of granularity (classes, packages,. . .)
- possibly interpreted as another instance of SOC
- examples include
  - choose the programming paradigm more suited to a certain context
  - create an interface to a class, and let others see/use just it
  - design an API layer on top of complex machinery
  - use a DSL instead of an API

Software engineering: principles V

AOC – anticipation of change

- evolution of software is inherent – differently from other engineering fields
- requirements are typically unstable, and this impacts design
- should always create the ground for flexible reactivity to changes
- sometimes, one needs to implement "ahead" for anticipation purposes
- examples include
  - move content of a class into an abstract class
  - make numbers and strings constants
  - encapsulate specific implementation choices
  - hide specific object creation behind public APIs

Software engineering: principles VI

GEN – generality

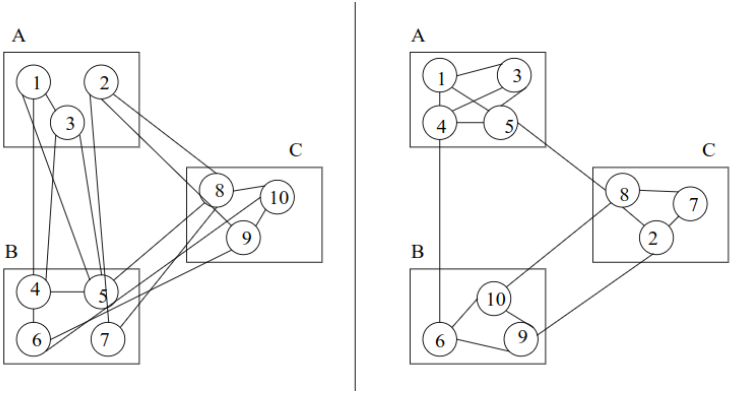
- solving a more general problem than the one at hand
- hence, promoting reuse to new situations
- this leads to a burden that somebody may not accept
- it is a principle coupling (similar to) AOC
- examples include
  - making specific implementation choices parameters
  - supporting more input cases than currently needed
  - supporting more working configurations than currently needed
  - combining two apparently unrelated functions into one

Software engineering: principles VII

INC – incrementality

- a quality for software processes at multiple levels of granularity
- it minimise the risk of coding on top of wrong code/assumptions
- it allows one to start addressing only the known requirements
- it allows ont to delay choices that would not be precise
- examples include
  - releasing software with incremental version ID
  - breaking a software development in week-wide "sprints" (as in SCRUM)
  - building software using TDD, with 20 minutes Red-Green-Refactor cycles

MOD explanation



Outline

1. Good design, abstraction, and process
2. Software quality criteria
3. Quality framework: problems and SE principles
4. Quality framework: OO paradigm
5. Software methodologies and the agile manifesto
6. Agile unit testing
7. A simple TDD-based example
8. Quality framework: (OO) design principles
9. Refactoring techniques and design patterns
10. Quality and the language paradigm

Object-orientation as a starting point

Paradigms and their impact on Software Engineering

- imperative (C), OO (Java), functional (Haskell), logic (Prolog)
- multiparadigm: Scala (OO+FP), Kotlin (OO+some FP), the many script langs

⇒ paradigms/languages/IDEs might significantly affect quality issues

How big is object-orientation?

- take Tiobe index: `http://www.tiobe.com/tiobe-index/`
- all top languages apart from *C* are OO
- the first language is Python ($\sim 23\%$)

OO, and Java, are essentially mainstream

- big and complex applications tend to use various languages...
- ...but the complex part is typically done in OO (Java, C#)
- most softeng books apply to OO langs (Java/C#)
- consider patterns (GoF), modelling languages (UML),...

Key elements of OO

Object

- an entity "living" in the runtime, with identity, state, and behaviour

Field

- a piece of an object state, holding an(other) object or value

Method

- handler of a request-reply "message" received/managed by objects
- a message affects state and produces a reply, concurrency as an orthogonal issue

Interface

- the collection of public method signatures (shape of request/reply)
- they define a contract for objects, i.e., a type

Class

- code implementing an interface and concretising methods/fields
- a template to create objects with same type, structure, and behaviour
- typically include secondary aspects (constructors, static members, genericity)

Key relationships in OO

Dependency (use-a)

- a class/interface A uses a class/interface B

Association/aggregation/composition (has-a)

- when class A's instance holds a reference to class B's instance
- association: independent life of A/B, no ownership
- aggregation: independent life of A/B, A owns B
- composition: "death" relationship (if A dies, B dies)

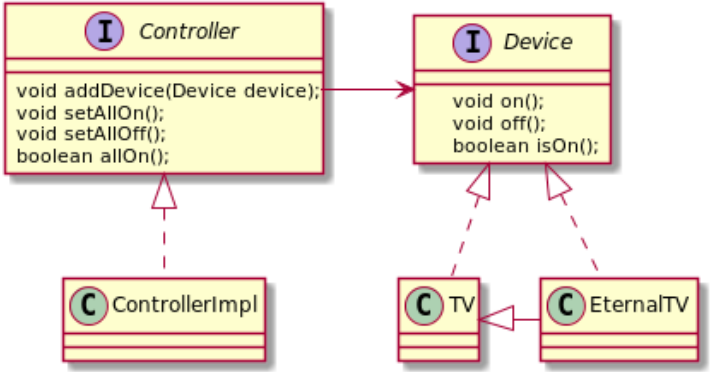
Inheritance (is-a)

- a class inherits all properties of another
- typically also with a subtyping relation, dynamic dispatching and casts

Implementation (is-a, conforms-to)

- a class implements all methods of an interface
- critically, with a subtyping relation

The Controller and Device example



Outline

1. Good design, abstraction, and process
2. Software quality criteria
3. Quality framework: problems and SE principles
4. Quality framework: OO paradigm
5. Software methodologies and the agile manifesto
6. Agile unit testing
7. A simple TDD-based example
8. Quality framework: (OO) design principles
9. Refactoring techniques and design patterns
10. Quality and the language paradigm

Steps of software development (not strictly in waterfall)

Requirement engineering (elicitation, analysis, specification)

- understanding the problem, by gathering requirements and organising/understanding/formalising them

Design

- defining the key elements of the solution: a software system
- by incremental refinement: from architecture to details

Implementation

- actual development: choosing technologies and coding

Post-implementation

- testing, deployment, maintenance: the resource-demanding phase!

Critics to the waterfall model

Instability of requirements

Requirements do change. In the waterfall this causes development to freeze until all stages are performed again.

Difficulty in foreseen detailed design

Ideally detailed designed (e.g., classes, interfaces and relationships) has to be done before implementation and testing. Unfortuntately, many design choices occur as a result of coding and testing.

Problems with late testing

Testing is key at any level of development. The more you wait the end, the more you write code on top of buggy abstractions that need to be changed.

Agile software development

The Agile manifesto initiative

- `http://agilemanifesto.org/`, 2001
- signed by many practicioners, the Agile Alliance
- parts of it deeply influenced software development
- started from criticising waterfall model
- defined by 12 principles
- key idea: do not refuse change, rather embrace it

In this course

- short introduction of principles here
- adopting main technique and philosophy throughout the course
- will later deepen methodologies, like Scrum

Agile software development: principles

Some principles—those that most affect coding

- highest priority is to satisfy the customer through early and continuous delivery of valuable software
- welcome changing requirements, even late in development—agile processes harness change for the customer's competitive advantage
- deliver working software frequently, from a couple of weeks to a couple of months, with a preference to the shorter time scale
- working software is the primary measure of progress
- continuous attention to technical excellence and good design enhances agility
- simplicity – the art of maximizing the amount of work not done – is essential

Agile programming as philosophy in this course

Key ideas

- break whole development in short SW pieces and periods
- in each period:
  1. identify and analyse the (new) requirements
  2. identify the main architectural decisions
  3. write key interfaces
  4. write deep tests (and prepare for agile, continuous testing)
  5. develop code until tests pass
  6. review/improve/refactor code before "merging upstream"
- generally promoting tools for continuous integration, throughout
- promote specific interactions of people over processes and tools

Outline

1. Good design, abstraction, and process
2. Software quality criteria
3. Quality framework: problems and SE principles
4. Quality framework: OO paradigm
5. Software methodologies and the agile manifesto
6. Agile unit testing
7. A simple TDD-based example
8. Quality framework: (OO) design principles
9. Refactoring techniques and design patterns
10. Quality and the language paradigm

Principles of test-driven development

Three Laws of TDD

1. You may not write production code until you have written a failing unit test.
2. You may not write more of a unit test than is sufficient to fail, and not compiling is failing.
3. You may not write more production code than is sufficient to pass the currently failing test.

Main methodology: three steps (10-20 minutes max)

1. Write a test that fails.
2. Get the test to pass.
3. Clean up any code added or changed in the prior two steps.

TDD impact

Why being so strict? To fully and really take advantage of...

- tests become a specification of intended behaviour, useful to understand goals and to document code
- tests dramatically reduce bug identification and resolution
- the ability of quickly testing gives freedom of refactoring

Method

- Traditional development cycle: Design, Code, Test
- TDD cycle (10-20mins): Test, Code, Design/Refactor

⇒ in practice, everybody stays in between...

In this course

- you have the unique chance of using it as thoroughly as you wish
- do it as thoroughly as you can, so you learn its pros and cons

Kinds of tests

Levels of software tests

- Unit test: isolating a single piece of code
- Integration test: considering interaction between components, possibly in target environments
- Functional test: examining code at the boundary of its public API (testing use cases)
- Stress test: considering actual workload and verifying the impact on resources
- Acceptance test: considering all aspects of user experience (requirements, ease, look)

Roles of unit tests

Different uses, possibly different tests

- For regression: to avoid changes to break existing code
- To measure progress: what is tested is what has been done
- For specificaton: to concisely capture code/API semantics
- To guide implementation: as in TDD
- To enforce solidity: e.g. by expressing all edge-cases
- To restore from legacy: to help refactoring old code

Clean tests

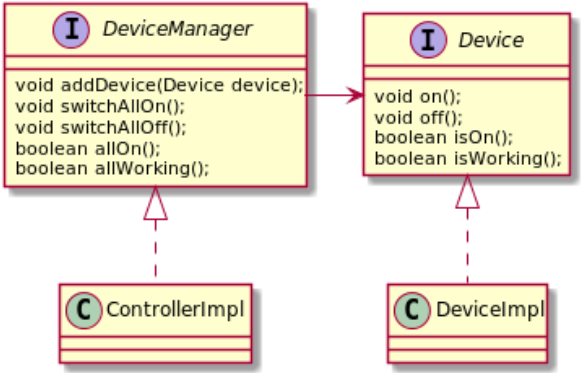
Recipes

- software for tests is a system per se, and it should be (very) clean, too
- readability is the key (clarity, simplicity, density of expression)
- there are several naming conventions, be consistent (e.g.: name the feature to be tested)
- adhere to build-operate-check pattern in each unit test
- strongly prefer one assert per test
- single concept per test (not one test per method!)
- tests should be: fast, independent, repeatable, self-validating, timely

Outline

1. Good design, abstraction, and process
2. Software quality criteria
3. Quality framework: problems and SE principles
4. Quality framework: OO paradigm
5. Software methodologies and the agile manifesto
6. Agile unit testing
7. A simple TDD-based example
8. Quality framework: (OO) design principles
9. Refactoring techniques and design patterns
10. Quality and the language paradigm

A reworked DeviceManager example



Interfaces – Device

```java
/**
 * An interface modelling a Device, which can be on/off.
 * It can also be non-working, meaning it is off and cannot be switched one.
 */
public interface Device {

    /**
     * Switches the device on
     */
    void on();

    /**
     * Switches the device off
     */
    void off();

    /**
     * @return whether the device is on
     */
    boolean isOn();

    /**
     * @return whether the device is actually working
     */
    boolean isWorking();
}
```

Interfaces – DeviceManager

```java
public interface DeviceManager {

    /**
     * Adds a device to be managed
     * @param device
     */
    void addDevice(Device device);

    /**
     * @return an immutable version of the list of added devices
     */
    List<Device> devices();

    /**
     * Switches on all the devices that are working
     */
    void switchAllOn();

    /**
     * Switches off all the devices
     */
    void switchAllOff();

    /**
     * @return whether all devices are switched on
     */
    boolean allOn();

    /**
     * @return whether all devices are working
     */
```

TDD

First implementation. . . steps:

1. Generate default implementations with the given names
2. Generate test classes, one per class to implement
3. Complete implementation

Notes

- Using JUnit
- Tests goes in the same package, but different folder
- Tests should be as good as code, possibly even better
- Tests have often greater size than tested code
- The result, in terms of code and test might not be perfect. . . one should refactor both in conjunction
- IntelliJ (configure test folder, use contextual menu)

Test Class – DeviceImplTest (1/3)

```java
public class DeviceImplTest {

    private static final int N_OF_ATTEMPTS = 10;
    private DeviceImpl device;

    @BeforeEach
    public void setUp() throws Exception {
        this.device = new DeviceImpl();
    }

    @Test
    public void isInitiallyOff() {
        assertFalse(this.device.isOn());
    }

    @Test
    public void canBeSwitchedOn() {
        this.device.on();
        assertTrue(this.device.isOn());
    }

    @Test
    public void canBeSwitchedOff() {
        this.device.on();
        this.device.off();
        assertFalse(this.device.isOn());
    }
```

Test Class – DeviceImplTest (2/3)

```java
    @Test
    public void canBeSwitchedOnTwice() {
        this.device.on();
        this.device.off();
        this.device.on();
        assertTrue(this.device.isOn());
    }

    @Test
    public void onIsIdempotent() {
        this.device.on();
        this.device.on();
        assertTrue(this.device.isOn());
    }

    @Test
    public void canBeSwitchedOffTwice() {
        this.device.on();
        this.device.off();
        this.device.on();
        this.device.off();
        assertFalse(this.device.isOn());
    }

    @Test
    public void canBeSwitchedMultipleTimes() {
        switchOnAndOffMultipleTimes();
        this.device.on();
        assertTrue(this.device.isOn());
    }
```

Test Class – DeviceImplTest (3/3)

```java
    private void switchOnAndOffMultipleTimes() {
        for (int i = 0; i < N_OF_ATTEMPTS; i++) {
            this.device.on();
            this.device.off();
        }
    }

    @Test
    public void initiallyWorks() {
        assertTrue(this.device.isWorking());
    }

    @Test
    public void typically Works() {
        switchOnAndOffMultipleTimes();
        assertTrue(this.device.isWorking());
    }

    @Test
    public void canStopWorking() {
        switchOnAndOffMultipleTimes();
        this.device.stopWorking();
        assertFalse(this.device.isWorking());
        this.device.on();
        assertFalse(this.device.isWorking());
        assertFalse(this.device.isOn());
    }
}
```

Class – DeviceImpl

```java
public class DeviceImpl implements Device {

    private boolean on = false;
    private boolean works = true;

    @Override
    public void on() {
        if (this.isWorking()) {
            this.on = true;
        }
    }

    @Override
    public void off() {
        this.on = false;
    }

    @Override
    public boolean isOn() {
        return this.on;
    }

    @Override
    public boolean isWorking() {
        return this.works;
    }

    public void stopWorking() {
        this.works = false;
        this.on = false;
    }
}
```

Test Class – DeviceManagerImplTest (1/2)

```java
public class DeviceManagerImplTest {

    public static final int SIZE = 3;
    private DeviceManager manager;
    private List<Device> devices;

    @BeforeEach
    public void init() {
        this.manager = new DeviceManagerImpl();
        this.devices = new ArrayList<>();
        for (int i = 0; i< SIZE; i++) {
            final Device d = new DeviceImpl();
            d.on();
            this.devices.add(d);
            this.manager.addDevice(d);
        }
    }

    @Test
    public void devices() {
        assertEquals(this.devices,this.manager.devices());
    }

    @Test
    public void switchAllOn() {
        this.manager.devices().get(0).off();
        this.manager.devices().get(SIZE-1).off();
        this.manager.switchAllOn();
        assertTrue(this.manager.allOn());
    }
```

Test Class – DeviceImplTest (2/2)

```java
    @Test
    public void switchAllOff(){
        this.manager.devices().get(SIZE-1).off();
        this.manager.switchAllOff();
        for (final Device device: this.manager.devices()){
            assertFalse(device.isOn());
        }
    }

    @Test
    public void allOn(){
        for (final Device device: this.manager.devices()){
            assertTrue(device.isOn());
        }
    }

    @Test
    public void allWorking(){
        for (final Device device: this.manager.devices()){
            assertTrue(this.manager.allWorking());
        }
    }

    @Test
    public void someNotWorking(){
        final DeviceImpl device = new DeviceImpl();
        device.stopWorking();
        this.manager.addDevice(device);
        assertFalse(this.manager.allWorking());
    }
```

Class – DeviceManagerImpl (1/2)

```java
public class DeviceManagerImpl implements DeviceManager {

    private List<Device> devices = new ArrayList<>();

    @Override
    public void addDevice(Device device) {
        this.devices.add(device);
    }

    @Override
    public List<Device> devices() {
        return Collections.unmodifiableList(this.devices);
    }

    @Override
    public void switchAllOn() {
        this.devices.forEach(Device::on);
    }

    @Override
    public void switchAllOff() {
        this.devices.forEach(Device::off);
    }
```

Class – DeviceManagerImpl (2/2)

```java
    @Override
    public boolean allOn() {
        for (final Device device: this.devices()){
            if (!device.isOn()){
                return false;
            }
        }
        return true;
    }

    @Override
    public boolean allWorking() {
        for (final Device device: this.devices()){
            if (!device.isWorking()){
                return false;
            }
        }
        return true;
    }
}
```

Tests as "speakable specification": a preview

Narrative style of tests in Scala

```
class EmptySetSpecification extends FlatSpec {
  "An empty Set" should "have size 0" in {
    Set.empty should have size (0)
  }

  it should "raise NoSuchElementException when head is called" in {
    an [NoSuchElementException] should be thrownBy (Set.empty.head)
  }
}
```

Outline

1. Good design, abstraction, and process
2. Software quality criteria
3. Quality framework: problems and SE principles
4. Quality framework: OO paradigm
5. Software methodologies and the agile manifesto
6. Agile unit testing
7. A simple TDD-based example
8. Quality framework: (OO) design principles
9. Refactoring techniques and design patterns
10. Quality and the language paradigm

Design principles: DRY

DRY: don't repeat yourself

- the bad practice: a piece of code to create seems similar to a previous one, so we cut & paste & modify
- this approach creates strong dependencies in software
- errors propagate and evolve without control
- the system becomes extremely rigid

⇒ duplication is the quintessential code smell

Facts

- can be remedied by proper interventions (refactoring)
- several tools can detect syntactic duplications with
- certain programming paradigms (e.g., FP) you are very aggressive in applying DRY

A simple example (almost not even OO)

```java
public class Logger {

    public static void logPrintString(final String s){
        System.out.format("[Print] %s\n",s);
    }

    public static void logPrintStrings(final String[] array){
        for (int i = 0; i < array.length; i++){
            System.out.format("[Print] %s\n",array[i]);
        }
    }
}
```

Applying DRY

```java
public class LoggerDry {

    public static void logPrintString(final String s){
        System.out.format("[Print] %s\n",s);
    }

    public static void logPrintStrings(final String[] array){
        for (int i = 0; i < array.length; i++){
            // reusing code, no duplication
            logPrintString(array[i]);
        }
    }
}
```

Design principles: KISS

KISS: keep it simple, stupid

- "most systems work best if they are kept simple rather than made complicated"
- a general design practice noted by the U.S. Navy in 1960
- never use complex solutions when easier ones are possible!
- when a complex solution emerges, think it twice!
- do not create problems that customers did not ask to be solved

Variants

- Occam's razor: "The simpler solution tend to be the right one"
- Leonardo da Vinci's: "Simplicity is the ultimate sophistication"
- Mies Van Der Rohe's: "Less is more"
- Bjarne Stroustrup's: "Make Simple Tasks Simple!"
- Antoine de Saint Exupéry's: "Perfection is reached when there's nothing left to take away"
- Einstein: "Everything should be made as simple as possible, but no simpler"

Applying KISS: getting rid of a tricky API

```java
public class LoggerKiss {

    public static void logPrintString(final String s){
        // avoiding a tricky and less known API
        System.out.println("[Print] "+s);
    }

    public static void logPrintStrings(final String[] array){
        // using for comprehension instead of explicit indexing
        for (final String s: array){
            logPrintString(s);
        }
    }
}
```

A proposed solution: the SOLID principles

5 cornerstones of good design/programming, generalising OO

- SRP: Single responsibility principle
  - ⇒ "A component should have one, and only one reason to change"
- OCP: Open/closed principle
  - ⇒ "Make components open for extension, but closed for modification"
- LSP: Liskov's substitutability principle
  - ⇒ "Extended code must be substitutable for their base version"
- ISP: Interface segregation principle
  - ⇒ "Make fine grained abstractions that are client specific"
- DIP: Dependency inversion principle
  - ⇒ "Depend on abstractions, not on concretions"

General facts

- in OO they mostly apply to classes (code) and interfaces (abstraction)
- but we will need to stay general and reason in a multi-paradigm way

SOLID principles: implications

Key ideas

- SRP: Single responsibility principle
  - keep components (classes, methods,. . .) small and coherent
  - "encapsulate away" what falls outside the key responsibility and/or could change
- OCP: Open/closed principle
  - allow access only to what is really needed
  - working code should never be modified, but integrated with new code
- LSP: Liskov's substitutability principle
  - make sure that extended/refined code is completely interchangable with the original one
- ISP: Interface segregation principle
  - of a component, different clients might perceive different abstractions requiring different views that one has to properly identify and provide
- DIP: Dependency inversion principle
  - clearly separate abstractions from concretions, and make only the former visible, avoid abstraction "jumps"

Applying SRP: moving a further logic responsibility away

```java
public class LoggerSrp {

    public static void logPrintString(final String s){
        // pushing the Prefix content responsibility out
        System.out.println(PrefixFormatter.printPrefix()+" "+s);
    }

    public static void logPrintStrings(final String[] array){
        for (final String s: array){
            logPrintString(s);
        }
    }
}

class PrefixFormatter {
    static String printPrefix() {
        return "[Print]";
    }
}
```

Applying OCP: making logPrintString unmodifiable

```java
public final class LoggerOcp {

    private final LogFormatter logFormatter;

    public LoggerOcp(LogFormatter logFormatter){
        this.logFormatter = logFormatter;
    }

    // This method now won't be likely changed..
    public void logPrintString(final String s){
        System.out.print(logFormatter.message(s));
    }

    public void logPrintStrings(final String[] array){
        for (final String s: array){
            logPrintString(s);
        }
    }
}
// can be changed by extension, instead of modification
class LogFormatter {
    protected String printPrefix() {
        return "[Print]";
    }
    public final String message(String s) {
        return printPrefix()+" "+s;
    }
```

Applying ISP: using a generalised client-specific interface

```java
public final class LoggerIsp {

    private final LogFormatter logFormatter;

    public LoggerIsp(LogFormatter logFormatter){
        this.logFormatter = logFormatter;
    }

    // This method now won't be likely changed..
    public void logPrintString(final String s){
        System.out.print(logFormatter.message(s));
    }

    // Using a general iterable.. not working for arrays actually
    // can now pass: List, Set, or Arrays.asList(array)
    public final void logPrintStrings(final Iterable<String> strings){
        for (final String s: strings){
            logPrintString(s);
        }
    }
}
```

Applying DIP: striving for depending on abstractions

```java
public class LoggerDip {

    // Abstracting System.out into an interface for others..
    interface Console {
        void print(String s);
        void println(String s);
    }
    // Abstracting LogFormatter into an interface for others..
    interface Formatter {
        String message(String s);
    }

    private final Formatter formatter;
    private final Console console;

    public LoggerDip(Formatter formatter, Console console){
        this.formatter = formatter;
        this.console = console;
    }

    public void logPrintString(final String s){
        console.print(formatter.message(s));
    }

    public void logPrintStrings(final Iterable<String> strings){
        for (final String s: strings){
            logPrintString(s);
        }
    }
}
```

Applying LSP: enforcing semantic substitutability

```java
public class LoggerLsp {

    interface Console {
        void print(String s);
        void println(String s);
    }
    //...
}
// What if the implementor messes with print/println?
// ...attempting at enforcing semantic substitutability
abstract class AbstractConsole implements LoggerLsp.Console {
    public final void println(String s) {
        this.print(s);
        this.lineFeed();
    }
    protected abstract void lineFeed();
}
class SystemOutConsole extends AbstractConsole {

    @Override
    public void print(String s) {
        System.out.print(s);
    }
    @Override
    protected void lineFeed() {
        System.out.println();
    }
}
```

Outline

1. Good design, abstraction, and process
2. Software quality criteria
3. Quality framework: problems and SE principles
4. Quality framework: OO paradigm
5. Software methodologies and the agile manifesto
6. Agile unit testing
7. A simple TDD-based example
8. Quality framework: (OO) design principles
9. Refactoring techniques and design patterns
10. Quality and the language paradigm

Refactoring code

Main facts

- it is the process by which a code/test is evolved without changing functionality
- the goal is to improve "quality", not to add functionality
- two objectives: enhance readability/generality or prepare for change
- availability of good tests is key to confidence in the outcomes

Basic techniques, from fine to coarse grained

- change name to variables, arguments, fields, methods and classes
- change signature (removing or combining arguments)
- create (private, then public) methods, static methods, constants
- substitute inheritance for delegation
- move code in new classes or base classes
- change constructors to static factories
- ...apply design patterns

Design Pattern

Design Patterns

- Idea: transmit experience on existing design to be reused *as is*
- They are not invented, rather extracted from analysing recurrent OO design solutions in diverse contexts
- Some are very famous, like those of the "Gang of Four" (GoF Patterns)
  - A book of paramount importance: "Design Patterns: Elements of Reusable Object-Oriented Software" E.Gamma, R.Helm, R.Johnson,J.Vlissides
  - 23 design patterns: Decorator, Singleton, Template Method, Observer,...
  - (It seems that "SW of big size use them all")
- Why using them?
  - They make your code more easily understandable, by just recalling them by name
  - They make code more flexible, namely, more open to change
  - They make your code much better organised

The 23 Patterns

Creational

- Class level: Factory Method
- Object level: Abstract Factory, Builder, Prototype, Singleton

Structural

- Class level: Adapter
- Object level: Adapter, Bridge, Composite, Decorator, Facade, Proxy

Behavioural

- Class level: Interpreter, Template Method
- Object level: Chain of Responsibility, Command, Iterator, Mediator, Memento, Flyweight, Observer, State, Strategy, Visitor

Proxy

Intent/motivation

Provide a surrogate (placeholder) for another object, to control accesses to it

Examples

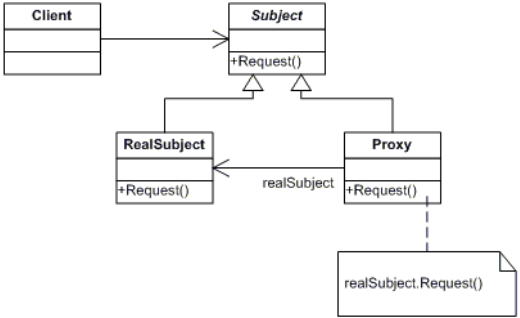
Control access to:

- optimise initialisation/use costs
- provide transparent access to a remote object
- provide limitations to its use
- provide limitations to its function (e.g., read-only collections)

Solution

- Create a wrapper that delegates to the controlled object, with same interface

Proxy: UML


Client
Subject
+Request()
RealSubject
+Request()
Proxy
+Request()
realSubject
realSubject.Request()

Proxy: consequences

Advantages

- Separates and encapsulates access logic
- Doesn't mess up the class that implements the base functionality
- Can be made transparent to the client
- Various stages of proxying can be used in cascade, and alternatively

Principles safeguarded by Proxy

- SRP: one class responsible for the core behaviour, one for access control
- DIP: client depends on interface, not on class, and this allows transparent use of the proxy

Strategy

Key motivation

A class `C` includes a portion of code, in the form of an algorithm/strategy (functional or non-functional), which is believed to easily change and/or later come in different versions

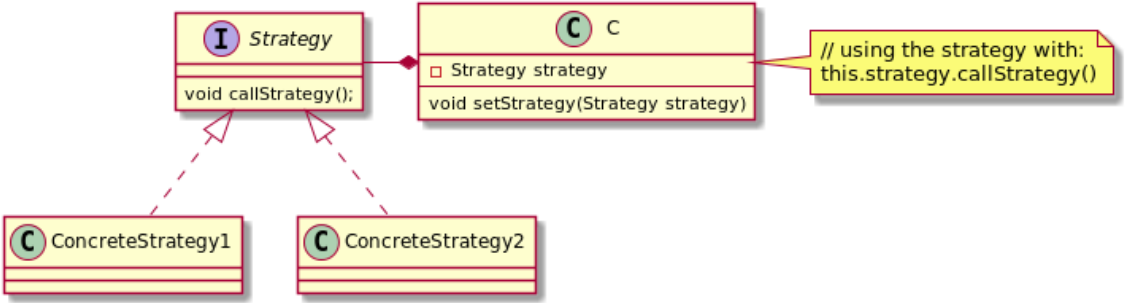
Solution

- the algorithm shape is captured by an interface `Strategy`, which can be implemented by different classes
- `C` is changed to take as input (in the constructor or a setter) an object of type `Strategy`, used where needed

Notes

- it is a quintessential (and somewhat natural) application of SRP and DIP
- anonymous classes (and lambdas) simplify adoption of this pattern
- Example: `java.util.Comparator`/`java.util.TreeSet`

Strategy: class diagram



Strategy: an example with devices

A possible change of requirements

- devices also have a name, with getter
- a manager can get all devices by name

$\Rightarrow$ want to prepare to a variety of different "search"

Solution

- a functional strategy is used to filter the devices to extract

Before: DeviceManagerImpl

```java
public class DeviceManagerImpl implements DeviceManager {

    private List<Device> devices = new ArrayList<>();

    @Override
    public void addDevice(Device device) {
        this.devices.add(device);
    }

    @Override
    public List<Device> devices() {
        return Collections.unmodifiableList(this.devices);
    }

    public List<Device> devicesByName(String s) {
        List<Device> list = new ArrayList<>();
        for (Device device: devices){
            if (device.getName().equals(s)){
                list.add(device);
            }
        }
        return Collections.unmodifiableList(list);
    }
```

After: DeviceManagerImpl

```java

    private List<Device> devices = new ArrayList<>();

    @Override
    public void addDevice(Device device) {
        this.devices.add(device);
    }

    @Override
    public List<Device> devices() {
        return Collections.unmodifiableList(this.devices);
    }

    public List<Device> filteredDevices(Predicate<Device> predicate) {
        //return this.devices.stream().filter(predicate).collect(Collectors.toList());
        List<Device> list = new ArrayList<>();
        for (Device device: devices){
            if (predicate.test(device)){
                list.add(device);
            }
        }
        return Collections.unmodifiableList(list);
    }
```

Template method

Key motivation

A class `C` includes a portion of code, in the form of an algorithm (functional or non-functional), which is believed to easily change and/or later come in different versions

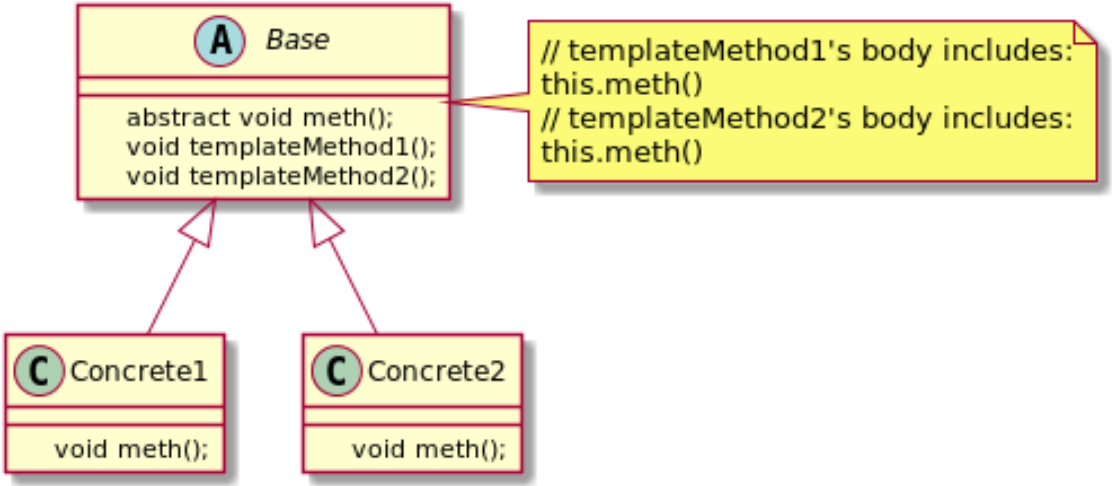
Solution

- the algorithm shape is captured by one or more abstract methods in the class, called by concrete (template) methods
- a concrete subclass of `C` must be created that implements the abstract methods, hence, the algorithm

Notes

- it is another natural application of SRP, with inheritance instead of delegation
- choice of an algorithm is here done at compile-time, not a run-time
- default implementation of methods in interfaces facilitates adoption of this patterns
- template method is almost mandatory in any class hierarchy
- Example: `java.io.InputStream`

Template method: class diagram



Template method: an example with devices

A possible change of requirements

- a new kind of device is added: a device which can break
- this new class would initially partially violate DRY principle

$\Rightarrow$ want to prepare to a variety of different device types

Solution

- template method is used in a base class to factor common code

Before: DeviceImpl

```java
public class DeviceImpl implements Device {

    private boolean on;
    private boolean working;
    private final String name;

    public DeviceImpl(String name) {
        this.on = false;
        this.working = true;
        this.name = name;
    }

    @Override
    public void on() {
        if (this.isWorking()) {
            this.on = true;
        }
    }

    @Override
    public void off() {
        this.on = false;
    }

    @Override
    public boolean isOn() {
        return this.on;
    }

    @Override
    public boolean isWorking() {
        return this.working;
    }

    void stopWorking(){ // non-public method
```

Before: BrokenDeviceImpl

```java
public class BrokenDeviceImpl implements Device {

    private boolean on;
    private boolean working;
    private final String name;

    public BrokenDeviceImpl(String name) {
        this.on = false;
        this.working = true;
        this.name = name;
    }

    @Override
    public void on() {
        if (this.isWorking()) {
            this.on = true;
        }
    }

    @Override
    public void off() {
        if (this.on && Math.random()>0.95){
            this.stopWorking();
        }
        this.on = false;
    }

    @Override
    public boolean isOn() {
        return this.on;
    }

    @Override
    public boolean isWorking() {
        return this.working;
    }

    void stopWorking(){ // non-public method
```

After: AbstractDevice

```java
public abstract class AbstractDevice implements Device {

    private boolean on;
    private boolean working;
    private final String name;

    public AbstractDevice(String name) {
        this.on = false;
        this.working = true;
        this.name = name;
    }

    @Override
    public void on() {
        if (this.isWorking()) {
            this.on = true;
        }
    }

    @Override
    public void off() {
        this.doWhenOff();
        this.on = false;
    }

    protected abstract void doWhenOff();

    @Override
    public boolean isOn() {
        return this.on;
    }

    @Override
    public boolean isWorking() {
        return this.working;
    }

    @Override
```

After: BrokenDeviceImpl and DeviceImpl

```java
public class BrokenDeviceImpl extends AbstractDevice {
    public BrokenDeviceImpl(String name) {
        super(name);
    }

    @Override
    protected void doWhenOff() {
        if (this.isOn() && Math.random()>0.95){
            this.stopWorking();
        }
    }
}
```

```java
public class DeviceImpl extends AbstractDevice {
    public DeviceImpl(String name) {
        super(name);
    }

    @Override
    protected void doWhenOff() {
    }
}
```

Decorator

Key motivation

An interface `I` has many implementations, to be flexibly mixed

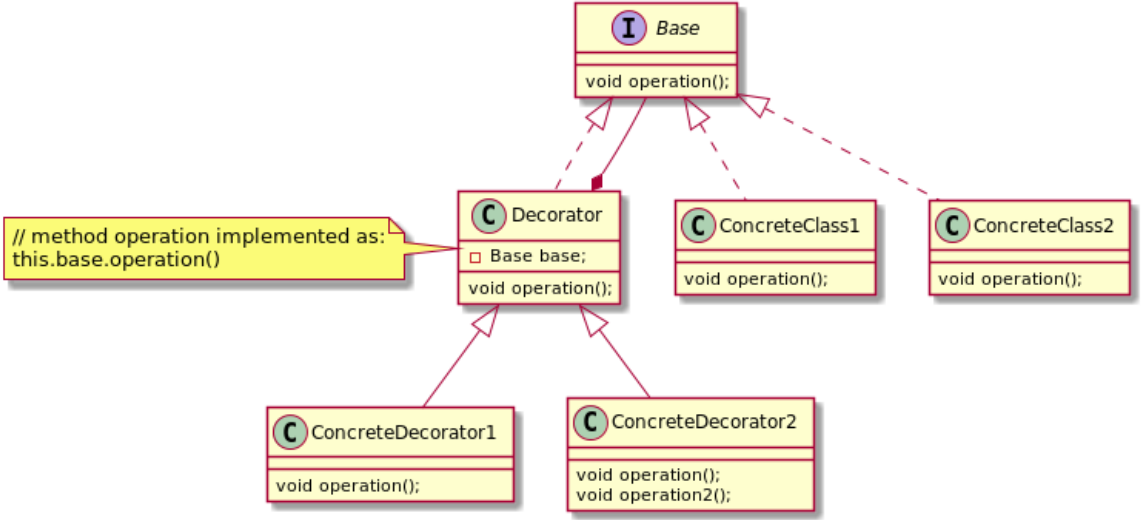
Solution

- provide a decorator class `C` that implements `I` and accepts an element of kind `I` (see strategy), to which most method calls are redirected, and which can be extended in different ways
- a chain of decorations can be created

Notes

- a mixture between strategy and template method
- the mix of algorithms can be selected at run-time
- default implementation of methods in interfaces facilitates adoption of this patterns
- Example: `java.util.stream.Stream`

Decorator: class diagram



Decorator: an example with devices

A possible change of requirements

- other than broken devices, one also adds devices with an enhanced interface, e.g., with a concept of variable luminosity

$\Rightarrow$ want to prepare to a variety of different devices, possibly with mix of features

Solution

- handle devices with decorations on top of a base class

Before: BrokenDeviceImpl

```java
public class BrokenDeviceImpl implements Device {

    private boolean on;
    private boolean working;
    private final String name;

    public BrokenDeviceImpl(String name) {
        this.on = false;
        this.working = true;
        this.name = name;
    }

    @Override
    public void on() {
    }

    @Override
    public void off() {
        this.on = false;
    }

    @Override
    public boolean isOn() {
        return this.on;
    }

    @Override
    public boolean isWorking() {
        return this.working;
    }

    @Override
    public String getName() {
        return this.name;
    }
```

Before: DeviceWithIntensityImpl

```java
public class DeviceWithIntensityImpl implements Device {

    private boolean on;
    private boolean working;
    private final String name;
    private int intensity;

    public DeviceWithIntensityImpl(String name) {
        this.on = false;
        this.working = true;
        this.name = name;
        this.intensity = 0;
    }

    @Override
    public void on() {
        if (this.isWorking()) {
            this.on = true;
        }
    }

    @Override
    public void off() {
        this.on = false;
    }

    @Override
    public boolean isOn() {
        return this.on;
    }

    @Override
    public boolean isWorking() {
        return this.working;
    }

    public void increaseIntensity(){
        this.intensity = this.getIntensity() + 1;
```

After: BrokenDeviceImpl

```java
public class BrokenDeviceImpl implements Device {


    private final Device device;

    public BrokenDeviceImpl(Device device) {
        this.device = device;
    }

    public void on() {
        device.on();
    }

    public void off() {
        device.off();
    }

    public boolean isOn() {
        return device.isOn();
    }

    public boolean isWorking() {
        return device.isWorking();
    }

    public String getName() {
        return device.getName();
    }
}
```

After: DeviceWithIntensityImpl

```java
public class DeviceWithIntensityImpl implements Device {

    private final Device device;
    private int intensity;

    public DeviceWithIntensityImpl(Device device) {
        this.device = device;
    }

    public void increaseIntensity(){
        this.intensity = this.getIntensity() + 1;
    }

    public int getIntensity() {
        return intensity;
    }

    public void on() {
        device.on();
    }

    public void off() {
        device.off();
    }

    public boolean isOn() {
        return device.isOn();
    }

    public boolean isWorking() {
        return device.isWorking();
    }

    public String getName() {
        return device.getName();
    }

}
```

After: Launcher

```java
public class Launcher {

    public static void main(String[] args) {
        Device d1 = new DeviceImpl("A");
        Device d2 = new DeviceWithIntensityImpl(new DeviceImpl("A"));
        Device d3 = new BrokenDeviceImpl(new DeviceImpl("A"));
        Device d4 = new DeviceWithIntensityImpl(
                            new BrokenDeviceImpl(
                                    new DeviceImpl("A")));
    }
}
```

Adapter

Key motivation

A class `C` contains a legacy behaviour that needs to be adapted to fit an interaface `I` someone expects

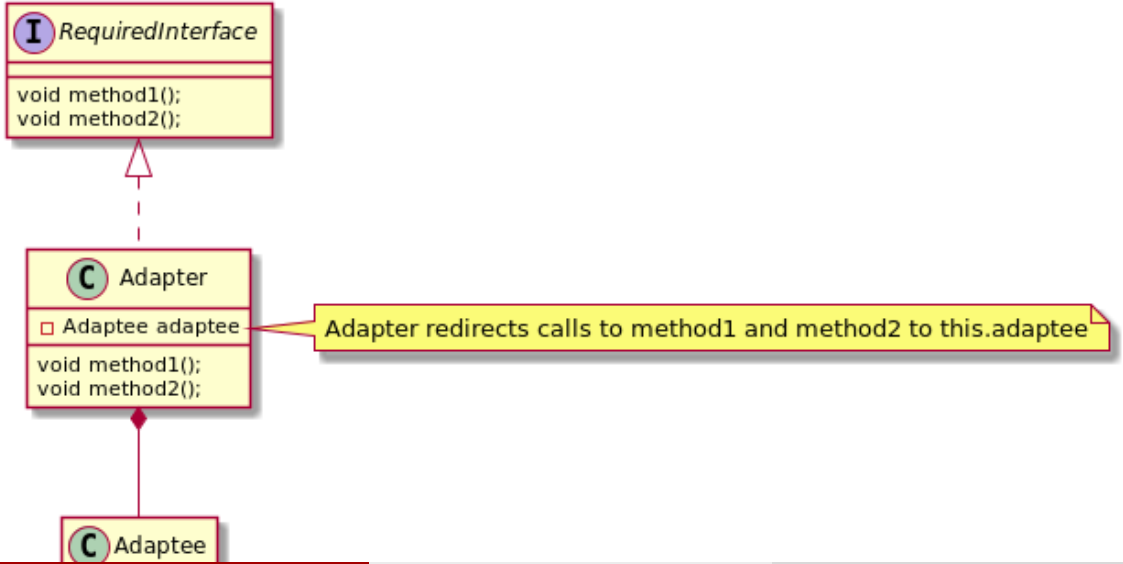
Solution

- provide a thin adapter class `A` implementing `I`, delegating all the needed work to `C`

Notes

- this pattern is needed since later implementing an interface is not feasible
- adaptation is often needed to overcome lack of design
- Example: `java.util.Arrays.asList`
- Example: in DIP, you create an interface someone has later to adapt to
- Example: you can wrap some bad legacy code tou need to use

Adapter: class diagram



Adapter: an example with devices

A possible change of requirements

- a `Client` class needed to work with a device manager before knowing its actual interface, namely, in a legacy way
- it assumed the existence of a `MyDeviceManagerInterface`

⇒ need to connect this client with `DeviceManager`

Solution

- following OCP we do not want to change `Client`
- an adpter is developed to fill the gap between `MyDeviceManager` and `DeviceManager`

Before: MyDeviceManager and Client

```java
public interface MyDeviceManager {

    void addSomeDevices(List<Device> device);

    void switchOn(int position);

    void switchOff(int position);

}
```

```java
public class Client {

    final MyDeviceManager manager;

    public Client(MyDeviceManager manager) {
        this.manager = manager;
    }

    void prepare(){
        this.manager.addSomeDevices(Arrays.asList(new DeviceImpl("A"),
                                                  new DeviceImpl("B")));
        this.manager.switchOn(0);
        this.manager.switchOff(1);
    }
}
```

After: MyDeviceManagerAdapter

```java
public class MyDeviceManagerAdapter implements MyDeviceManager {

    final private DeviceManager manager;

    public MyDeviceManagerAdapter(DeviceManager manager) {
        this.manager = manager;
    }

    @Override
    public void addSomeDevices(List<Device> list) {
        for (Device dev: list){
            this.manager.addDevice(dev);
        }
    }

    @Override
    public void switchOn(int position) {
        this.manager.devices().get(position).on();
    }

    @Override
    public void switchOff(int position) {
        this.manager.devices().get(position).off();
    }
}
```

Abstract Factory

Key motivation

The decision on which versions of a group of classes/interfaces `C1`,..,`Cn` should actually be instantiated has to be taken dynamically

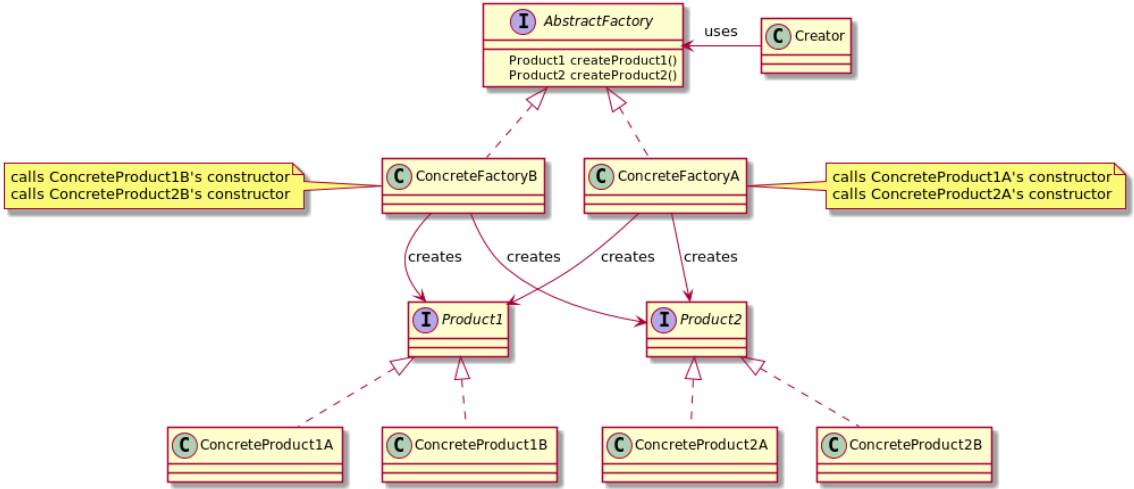
Solution

- provide a factory interface `IF` with n factory methods, and `I1`,. . . ,`In` interfaces for those classes; an actual implementation of `IF` defines which concrete classed to use

Notes

- recall that the construction logic should be separated from the usage logic
- this patterns helps in making code completely independent from actual implementation
- Factory Method (where `IF` is an actual class and $n = 1$) is a subcase, very used throughout libraries
- Example: `java.util.Stream`

Abstract Factory: class diagram



Abstract Factory: an example with devices

A possible change of requirements

- there's a proliferation in the kind of types
- the logics by which we decide what to instantiate can vary dynamically

$\Rightarrow$ need to make transparent the concrete class to create

Solution

- create a factory for devices

After: DeviceFactory

```java
public interface DeviceFactory {

    Device createStandardDevice(String name);

    Device createBrokenDevice(String name);

    Device createRandomDevice(String name);

}
```

```java
public final class DeviceFactoryImpl implements DeviceFactory {

    @Override
    public Device createStandardDevice(String name) {
        return new DeviceImpl(name);
    }

    @Override
    public Device createBrokenDevice(String name) {
        return new BrokenDeviceImpl(name);
    }

    @Override
    public Device createRandomDevice(String name) {
        return Math.random()>0.5 ? createStandardDevice(name) : createBrokenDevice(name);
    }
}
```

```java
    public static void main(String[] args) {
        DeviceFactory factory = new DeviceFactoryImpl();
        Device d1 = factory.createStandardDevice("A");
        Device d2 = factory.createBrokenDevice("A");
```

Builder

Key motivation

A class `C` includes too much construction logic, with many/alternative inputs, default values, sequential initialisation actions, and so on

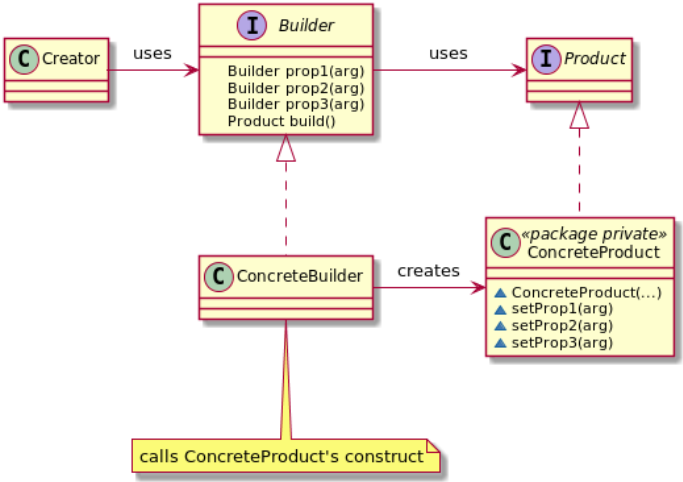
Solution

- provide a companion builder class `Builder`, whose objects have the sole responsability of the construction logic, and eventually build one object
- a fluent interface is typically used to facilitate one-statement construction
- details of the concrete class to build are often hidden inside (package or inner class)

Notes

- recall that very few constructors and arguments are tolerated with clean code
- Example: `java.util.Stream.Builder`, `java.lang.StringBuilder`

Builder: class diagram



Before: DeviceImpl

```java
public class DeviceImpl implements Device {

    private boolean on;
    private boolean working;
    private final String name;
    private final String description;

    public DeviceImpl() {
        this("Device");
    }

    public DeviceImpl(String name) {
        this(name,"");
    }

    public DeviceImpl(String name, String description) {
        this.description = description;
        this.on = false;
        this.working = true;
        this.name = name;
    }

    @Override
    public void on() {
        if (this.isWorking()) {
            this.on = true;
        }
    }

    @Override
    public void off() {
        this.on = false;
    }

    @Override
```

After: DeviceBuilder

```java
public class DeviceImplBuilder {
    private String name = "Device";
    private String description = "";

    public DeviceImplBuilder setName(String name) {
        this.name = name;
        return this;
    }

    public DeviceImplBuilder setDescription(String description) {
        this.description = description;
        return this;
    }

    public DeviceImpl createDeviceImpl() {
        return new DeviceImpl(name, description);
    }
}
```

```java
public class Launcher {

    public static void main(String[] args) {
        Device d1 = new DeviceImplBuilder().setName("A")
                                           .setDescription("a nice device")
                                           .createDeviceImpl();
        Device d2 = new DeviceImplBuilder().setName("A").createDeviceImpl();
        Device d3 = new DeviceImplBuilder().createDeviceImpl();
    }
}
```

Outline

1. Good design, abstraction, and process
2. Software quality criteria
3. Quality framework: problems and SE principles
4. Quality framework: OO paradigm
5. Software methodologies and the agile manifesto
6. Agile unit testing
7. A simple TDD-based example
8. Quality framework: (OO) design principles
9. Refactoring techniques and design patterns
10. Quality and the language paradigm

Quality and OO

The standard case

- software quality is considered a problem only in big projects
- big projects are typically developed using OO mainstream langs

In reality

1. quality is to be achieved independently of the language used
2. having the proper language/paradigm can be a tool to achieve quality...

⇒ ...the right abstraction might be better captured

Language paradigms

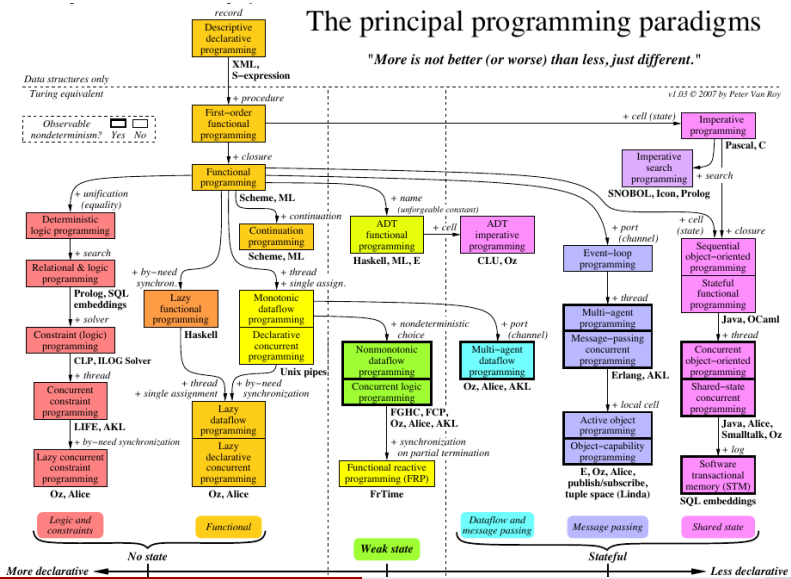
Definition

A programming paradigm is a fundamental style of computer programming, that serves as a school of thoughts

Languages, Paradigms, and Concepts

- a language adhere to one of few paradigms (sometimes two)
- paradigms are rooted in few concepts and computational mechanisms
- mechanisms are
  - primitive expressions: simplest entities manipulated
  - means of combination: building compound elements out of simpler ones
  - means of abstraction: naming and manipulating compound elements as units

Many languages, few paradigms



Main paradigms: Imperative languages

Motto

First do this, then do that

Elements

- computing as executing statements that change a program state
- structured programming constructs (for, while) help control
- known to be the most "natural" approach for beginners
- inspired by hardware aspects
- concepts: instructions, procedures, modules
- also the starting point for OO

Examples

- Fortan, Algol, Pascal, Basic, C

Main paradigms: Object-oriented languages

Motto

Send a message to an object to simulate a real-world event

Elements

- computing as letting a bunch of objects exchange messages
- objects encapsulate data and behaviour
- known to be the most "powerful" general-purpose approach
- concepts: objects, classes, methods, fields
- typically, method bodies following the imperative style

Examples

- C++, Java, C#

Main paradigms: Functional languages

Motto

Evaluate an expression and use the resulting value for something

Elements

- computing as applying functions without side effects
- recursion used instead of iteration
- known to be the most "elegant" approach for pros
- inspired by mathematics, and lambda-calculus
- later mixed with OO (see Scala)

Examples

- Haskell, F#, Erlang, ML, Scheme, Lisp

Main paradigms: Logic languages

Motto

Answer a computational question via search for a solution

Elements

- computing as checking validity of a relationship among data
- based on axioms, rules, matching, and logic inference
- known to greatly work in selected cases (related to AI)
- inspired by first-order logic and theorem proving

Examples

- Prolog, Datalog

The general case of declarative languages

Declarative languages

They express the logic of computation abstracting away from the specific control-flow, namely, focussing on what rather than how

Paradigms

- functional paradigm: hides the order of evaluation of subsexpression
- logic paradigm: hides the search startegy

Advantages for quality

- much more concise and elegant
- give a feeling of actual computational steps emerging "magically"

Multi-paradigm approaches

Idea

- identifying languages or frameworks allowing to use different paradigms for different parts of a system, leveraging contamination between paradigms
- this historically leads to much more complex languages
- experience and improvements might create a breakthrough

Modern OO languages

- Java and C# have primitive mechanisms of functional programming

The case of Scala

- a full OO language with very advanced type system and eye-candy mechanisms
- first-order functions exist as well

Other approaches

- integration of Prolog and Java in tuProlog

Domain-specific Languages (DSLs)

DSL

languages specifically-tailored to particular domains, namely, not general-purpose

Facts

- often used in combination with mainstream languages
- recently re-emerged thanks to nice frameworks to build DSLs (e.g., Xtext)
- turning an API into a DSL, when succeeding, raises abstraction
- inside every system there's a small language struggling to get out

Many examples

- description languages (for games, configurations, meta-compilers, softeng)
- specific scripting languages (shell scripts)

Concurrency

The problem of concurrency

- concurrency seems to badly fit all main paradigms
- traditional mechanisms, like threads, are considered too low-level
- recent advances in multi-core architectures made virtually any non-trivial application a concurrent-one
- concurrency is often intertwined with distribution

Attacking the problem

- promoting high-level (typically functional-inspired) libraries
- advancing the OO layer with new paradigms (e.g. with actors, agents, streams)

In this course

- the problem is completely delegated to the "Concurrent and Distributed Programming" side
- we shall assume advanced libraries give us the abstractions we need